

VIETNAMESE - GERMAN UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE



COMPUTER NETWORK 2
BUILD AN FTP CLIENT IN JAVA

Lecturer: Dr. Trương Đình Huy

Student: Nguyễn Phúc Nhân - 10423083

1 Introduction

1.1 Project Goal

The goal of this project is to implement a fully functional FTP client using Java. This client can interact with standard public FTP servers and perform fundamental file transfer operations. The program was written using core Java packages: `java.io.*`, `java.net.*`, and `java.util.*`. A graphical user interface was also developed using the Java Swing library.

1.2 Requirements

This FTP client satisfies the following functional requirements:

- It connects to an FTP server through port 21 and closes the connection properly.
- It supports both anonymous login and authenticated login.
- It retrieves the current working directory `PWD` and change directories `CWD`.
- It lists remote directory contents using passive mode `PASV + LIST`.
- It downloads files from the server in binary mode `PASV + RETR`.
- It uploads local files to the server in binary mode `PASV + STOR`.
- It manages remote files and directories like delete files, create and remove directories.

2 Architecture Design

The FTP client was implemented in Java, strictly following to the project requirements by using only standard libraries such as `java.net.*` for networking, `java.io.*` for stream processing and `javax.swing.*` for the GUI. The architecture is cleanly divided into two main layers: the Core FTP Logic layer and the GUI layer.

2.1 Core Architecture

The core logic is structured to separate socket communication from FTP commands:

- **FTPConnection.java:** This class manages the TCP Control Connection on port 21. It initializes the socket, handles raw input and output streams, processes correctly multi-line responses from the server such as lines ending with a hyphen like `220-`. It also implements a logging listener to hide the passwords while forwarding command sequences to the UI.
- **FTPClient.java:** This class is an abstraction layer for the underlying `FTPConnection.java`. It implements specific FTP operations like handling user authentication, navigating directory, and constructing commands for file manipulation.

- **FTPResponse.java and FTPException.java:** These 2 classes work together to ensure robust error handling. The **FTPResponse.java** parses raw server strings into numeric codes and textual messages. If an error code is detected with code ≥ 400 , an **FTPException.java** is thrown, allowing the upper layers to catch and display meaningful error dialogs to the user.

2.2 GUI and Multithreading

The GUI is built with Java Swing, it is divided into 4 specialized panels, all coordinated by a central frame:

- **MainFrame.java:** This is the main window of the application. It acts as a brain that coordinates all the interfaces and logic components together, utilizing a **JSplitPane** to properly arrange the layout.
- **LoginPanel.java:** This panel is located at the top of the UI, it contains a form for entering connection information such as host, port, user, password, along with a connect/disconnect button.
- **LocalPanel.java:** This panel displays on the left side, it features the local file browser (representing the user's computer).
- **RemotePanel.java:** This panel is on the right side, it features the remote file browser from the FTP server's side.
- **LogPanel.java:** This panel is on the bottom, it displays the real-time protocol log for monitoring server responses.

To ensure the GUI remains responsive, multithreading is used. All network-related operations like connecting, uploading, downloading, listing are executed in separate background **Threads**. Once an operation completes, the **SwingUtilities.invokeLater()** method is used to safely push UI updates like refreshing the **JTable** or appending to the log area back to the Event Dispatch Thread.

3 Protocol Flow

3.1 Passive Mode (PASV) and Data Connection

To handle data transfers such as **LIST**, **RETR** or **STOR**, the FTP Client creates a new Data Connection in Passive mode.

1. The client sends the **PASV** command over the Control Connection.
2. The server responds with **227 Entering Passive Mode (h1,h2,h3,h4,p1,p2)**.
3. The client parses this response to calculate the server's data IP address (**h1.h2.h3.h4**) and data port (**p1*256 + p2**).
4. The client opens a new TCP socket to this computed IP and port. This is the Data Connection.

3.2 Connect Flow

When the user clicks "Connect", the following sequence occurs:

1. The client opens a TCP socket to the server's IP address on port 21.
2. The server responds with a welcome message with a 220 code.
3. The client sends the **USER username** command.
4. The server responds with 331 Password required or 230 Logged in.
5. If 331 is received, the client sends **PASS password**.
6. The server responds with 230 Login successful.

3.3 File Download Flow

When the user selects a file and clicks "Download" button, the following sequence occurs:

1. The client sends **TYPE I** to switch to binary mode.
2. The server responds with 200 Switching to Binary mode.
3. The client sends the **PASV** command to request a passive data connection.
4. The server responds with 227 Entering Passive Mode (h1,h2,h3,h4,p1,p2) containing the IP and Port for the data connection.
5. The client opens a new TCP data socket to the IP and Port provided by the server.
6. The client sends the **RETR filename** command over the control connection.
7. The server responds with 150 Opening BINARY mode data connection.
8. The server transmits the file contents over the data connection and closes it when finished.
9. The server responds with 226 Transfer complete over the control connection.

3.4 File Upload Flow

When the user selects a local file and clicks "Upload" button, the following sequence occurs:

1. The client sends **TYPE I** to switch to binary mode.
2. The server responds with 200 Switching to Binary mode.
3. The client sends the **PASV** command to request a passive data connection.
4. The server responds with 227 Entering Passive Mode (h1,h2,h3,h4,p1,p2) containing the IP and Port for the data connection.

5. The client opens a new TCP data socket to the IP and Port provided by the server.
6. The client sends the **STOR filename** command over the control connection.
7. The server responds with 150 Ok to send data.
8. The client transmits the file contents over the data connection and closes it when finished.
9. The server responds with 226 Transfer complete over the control connection.

3.5 Directory Content List Flow

When the client wants to refresh the remote file browser by clicking "Refresh", the following sequence occurs:

1. The client sends the **PASV** command to request a passive data connection.
2. The server responds with 227 Entering Passive Mode (h1,h2,h3,h4,p1,p2) containing the IP and Port for the data connection.
3. The client opens a new TCP data socket to the IP and Port provided by the server.
4. The client sends the **LIST** command over the control connection.
5. The server responds with 150 Here comes the directory listing.
6. The server transmits the directory listing over the data connection and closes it when finished.
7. The server responds with 226 Directory send OK over the control connection.

3.6 Directory Change Flow

When the user double-clicks a remote folder or clicks "Back" button, the following sequence occurs:

1. The client sends **CWD directory-path** command.
2. The server responds with 250 Directory successfully changed.
3. The client sends the **PWD** command to retrieve the new absolute path.
4. The server responds with 257 current-path is the current directory. After this, the client automatically initiates a LIST flow to update the UI.

3.7 Directory Create Flow

When the user clicks "New Folder" button and enters a name, the following sequence occurs:

1. The client sends the **MKD directory-name** command.
2. The server responds with 257 directory-name created.

3.8 Directory Remove Flow

When the user selects a remote folder and clicks "Remove Folder" button, the following sequence occurs:

1. The client sends the `RMD directory-name` command.
2. The server responds with 250 Directory successfully removed.

3.9 File Delete Flow

When the user selects a remote file and clicks "Delete" button, the following sequence occurs:

1. The client sends the `DELE filename` command.
2. The server responds with 250 Delete operation successful.

3.10 Disconnect Flow

When the user clicks "Disconnect" button or closes the application, the following sequence occurs:

1. The client sends the `QUIT` command.
2. The server responds with 221 Goodbye.
3. The client safely closes the control TCP socket.

4 Execution and Test Commands

This project is built and managed using Apache Maven.

4.1 Build and Run Instructions

To compile the project and build the executable JAR file, navigate to the `FTPClient` directory and run the following Maven command:

```
1 mvn clean package
```

The application can be launched from the terminal using the `java -jar` command:

```
1 java -jar target/ftp-client-1.0.jar
```

4.2 Usage Example

I have tested the application with two different environments:

4.2.1 Test 1: Public FTP Server ftp.gnu.org

This test demonstrates connecting to a public read-only server to navigate and download files.

1. In the login panel, I entered the host `ftp.gnu.org`, port `21`, username `anonymous` and left the password blank.
2. I clicked the **Connect** button, the remote panel then displayed with the server's root directory files.

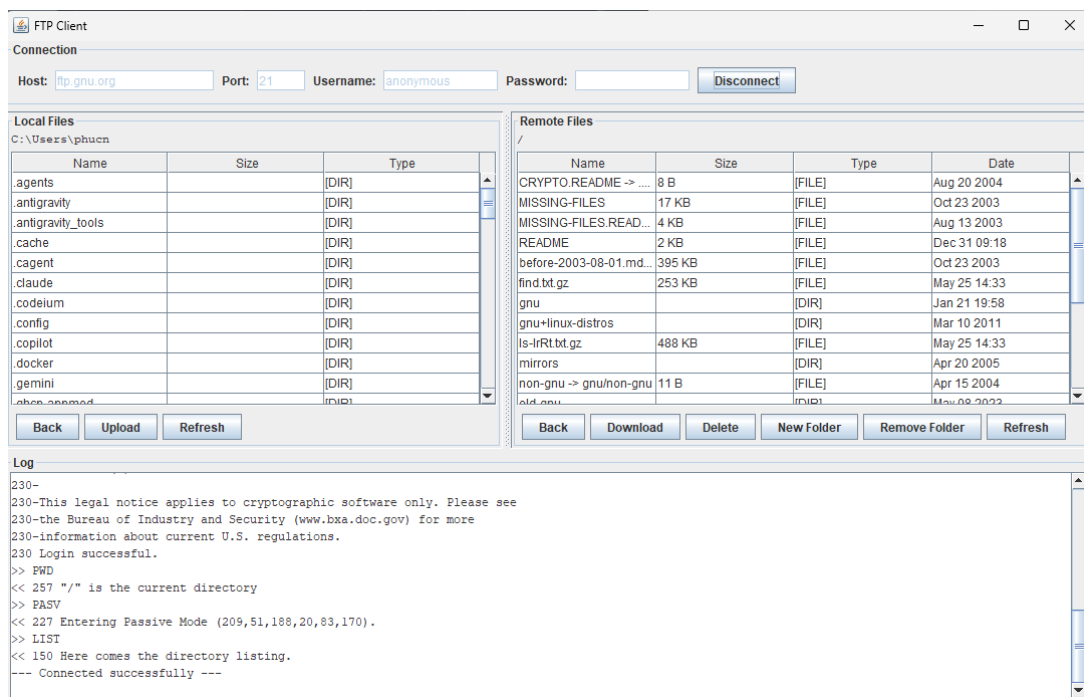


Figure 1: Connecting to ftp.gnu.org

3. I double-clicked the `gnu` folder in the remote panel to navigate into it.

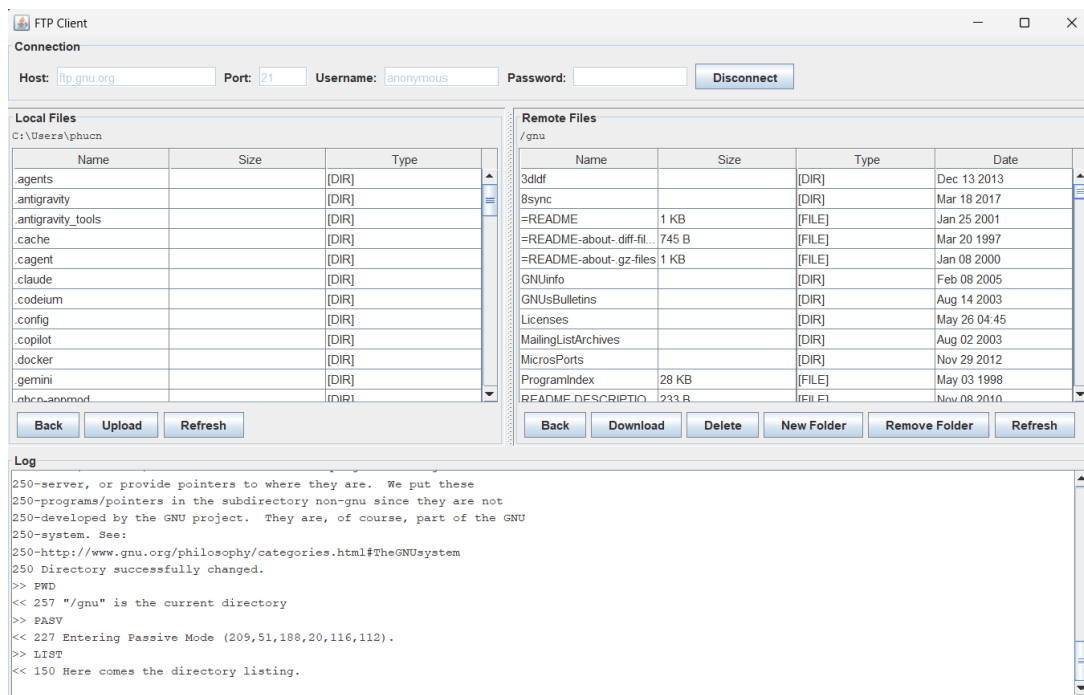


Figure 2: Navigating into a remote folder

- I clicked the **Back** button in the remote panel to return to the parent directory.

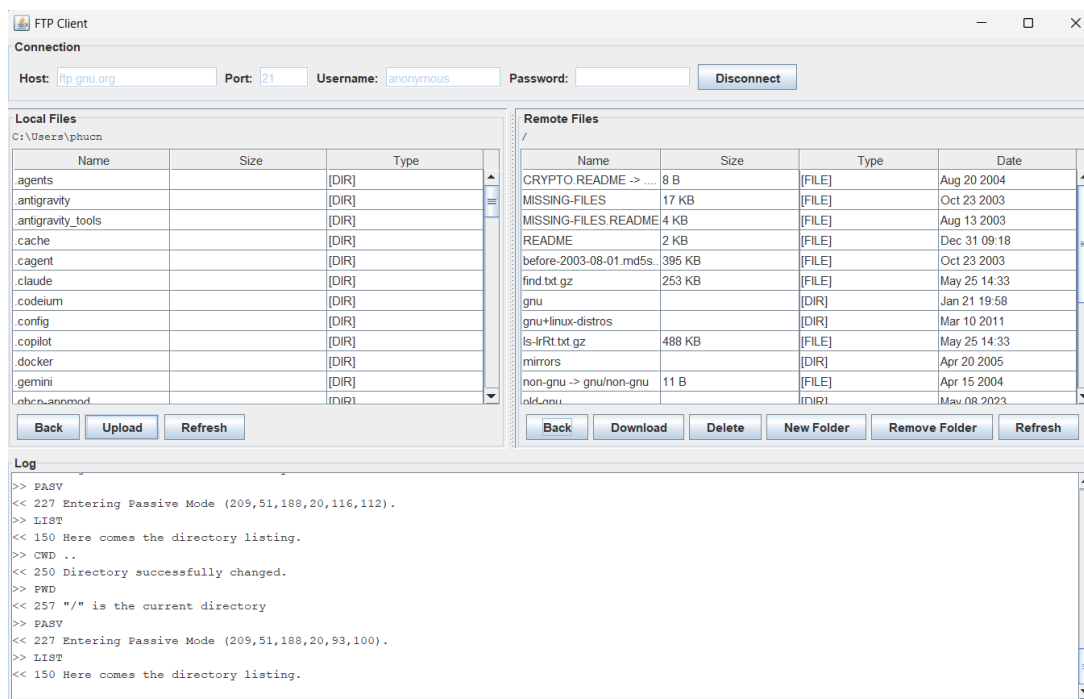


Figure 3: Using the Back button

- I selected a README file in the Remote Panel and clicked the **Download** button. The file was downloaded and appeared in the local panel.

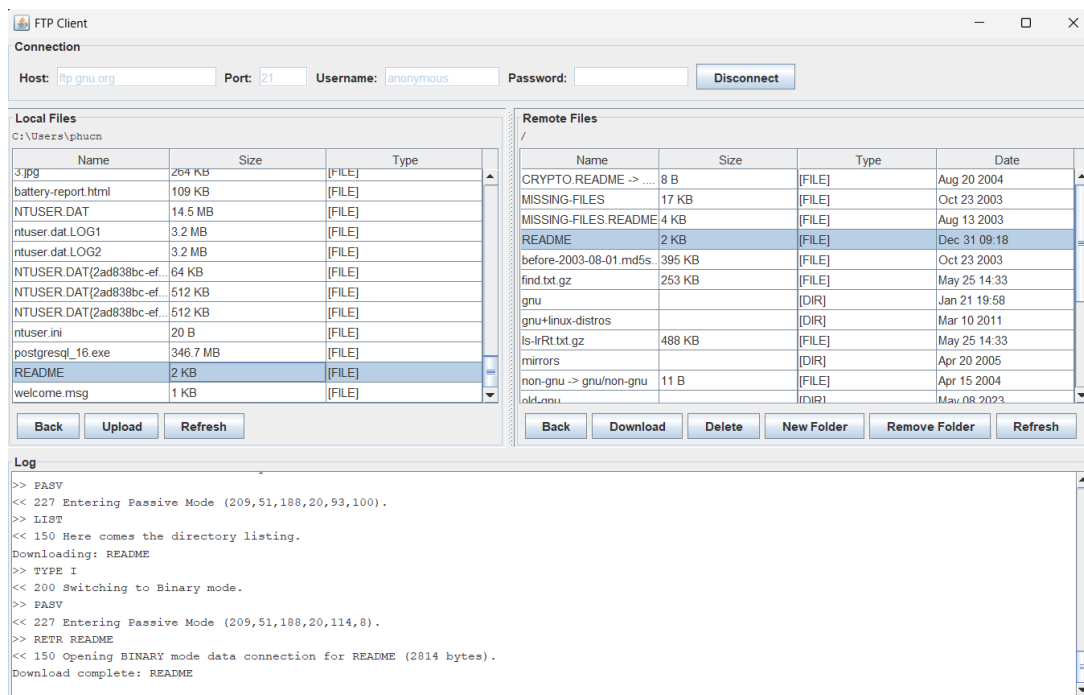


Figure 4: Downloading a file from the server

6. I clicked the **Refresh** button in the local panel to manually refresh the view and verify the downloaded file.
7. Finally, I clicked the **Disconnect** button to close the session.

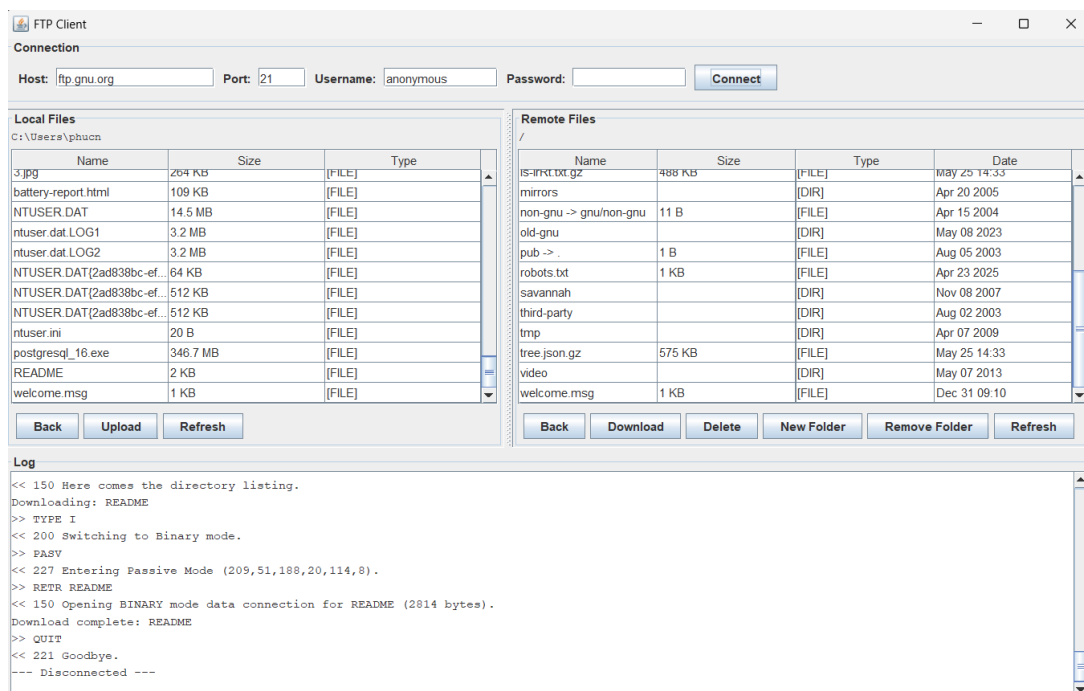


Figure 5: Disconnecting from the server

- I also clicked the **Upload**, **Delete**, **Remove Folder** and **New Folder** buttons but these actions were denied by the server with a 550 Permission denied error because it is a read only public server.

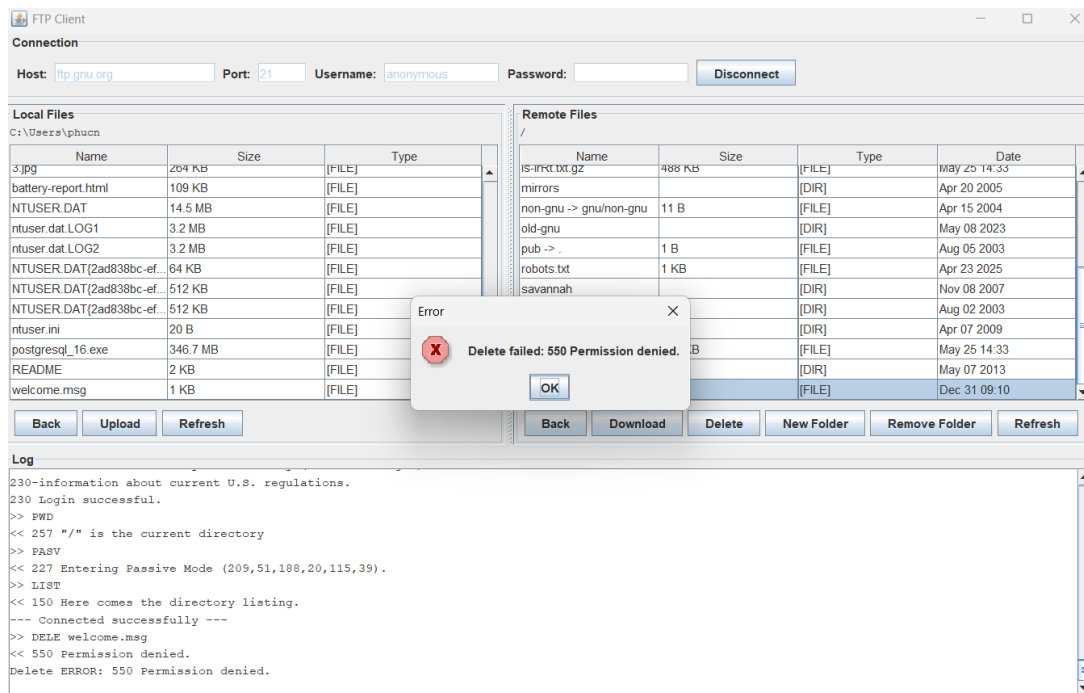


Figure 6: Trying other buttons

4.2.2 Test 2: Local FileZilla Server

This test demonstrates file manipulation operations like upload, create folder, delete using a locally hosted FileZilla Server with full permissions.

- First, I set up a local FileZilla Server on 127.0.0.1 with username `test`, password 1234 having full read and write privileges.

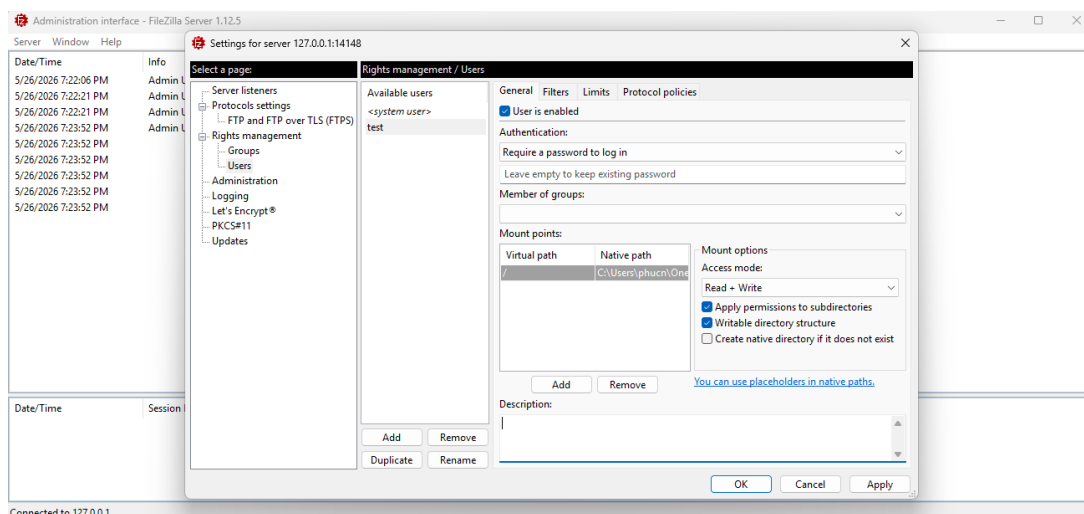


Figure 7: FileZilla Server configuration

2. In the login panel, I entered the host 127.0.0.1, port 21, username `test`, and password 1234.
3. I clicked the **Connect** button to establish the connection.

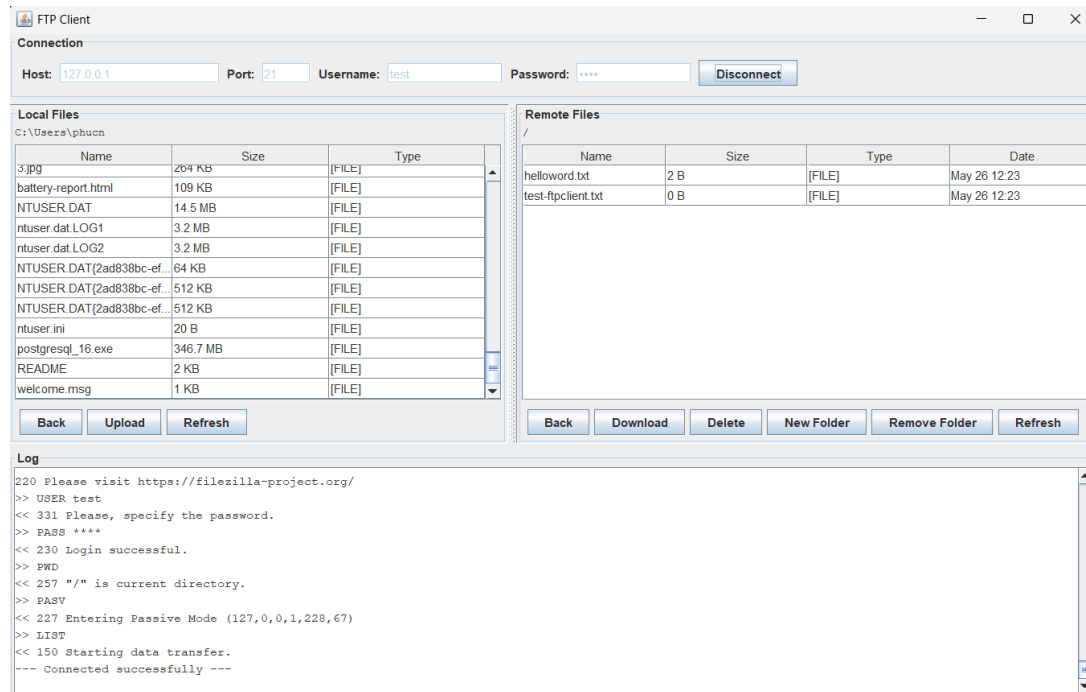


Figure 8: Connecting to local FileZilla Server

4. I selected `welcome.msg` file in the local panel and clicked the **Upload** button. The file was successfully transferred to the FTP server.

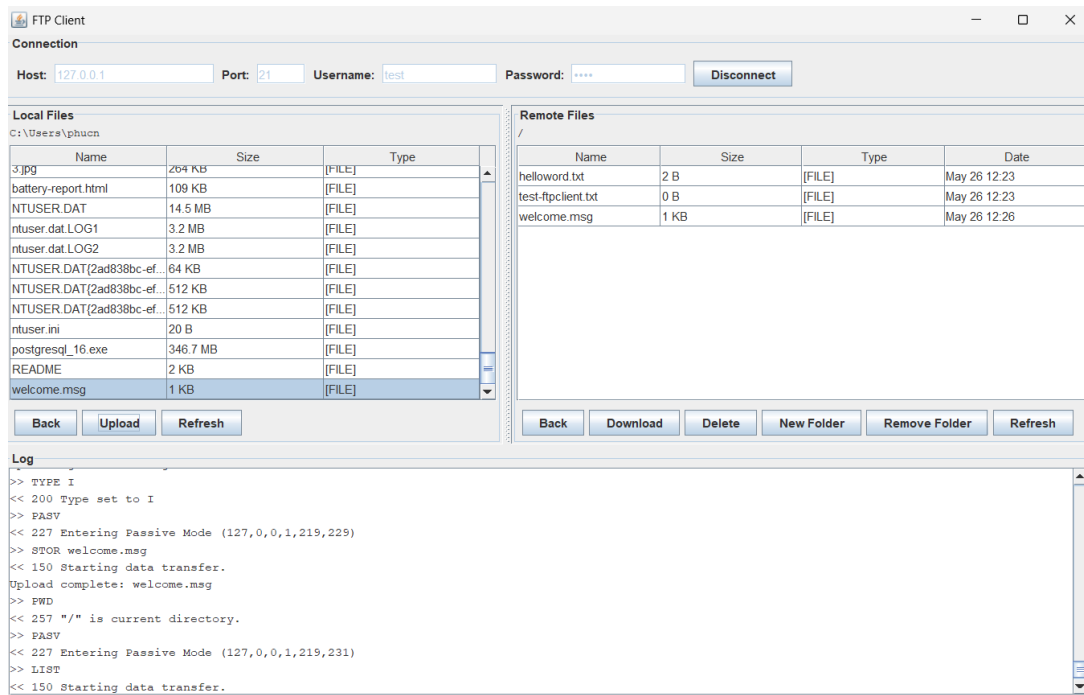


Figure 9: Uploading a file to the server

- I clicked the **Refresh** button in the remote panel to see the newly uploaded file appear in the remote directory.
- I clicked the **New Folder** button in the remote panel, entered the folder name test-newfolder in the dialog, and confirmed to create a new directory on the server.

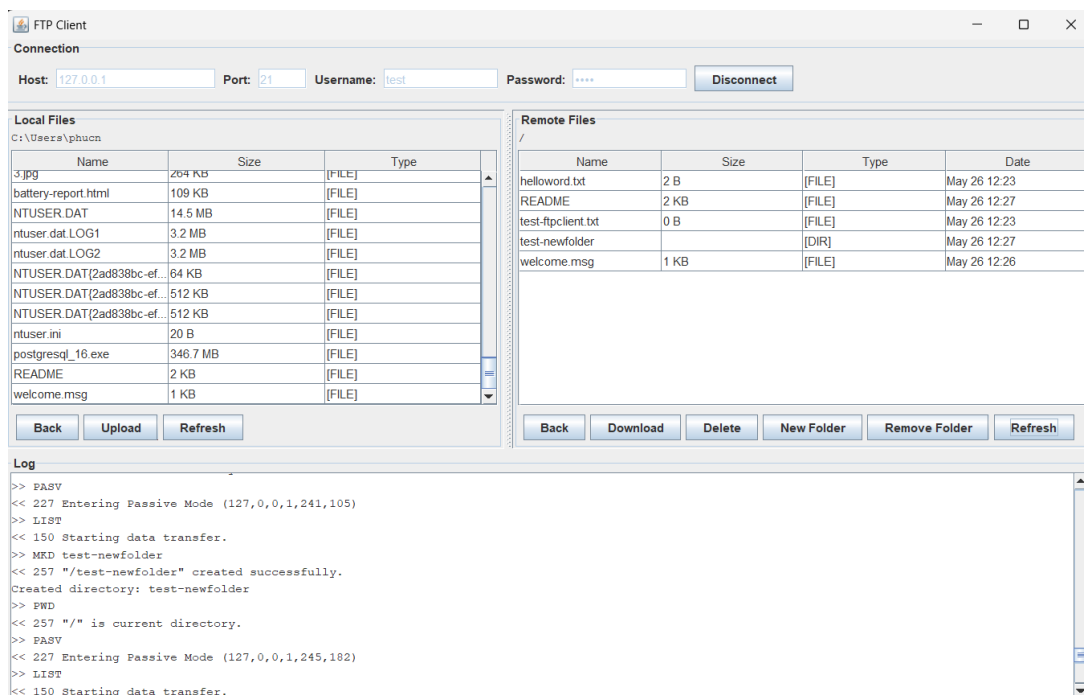


Figure 10: Creating a new remote directory

7. I selected the welcome.msg file in the remote panel and clicked the **Delete** button to remove it from the server.

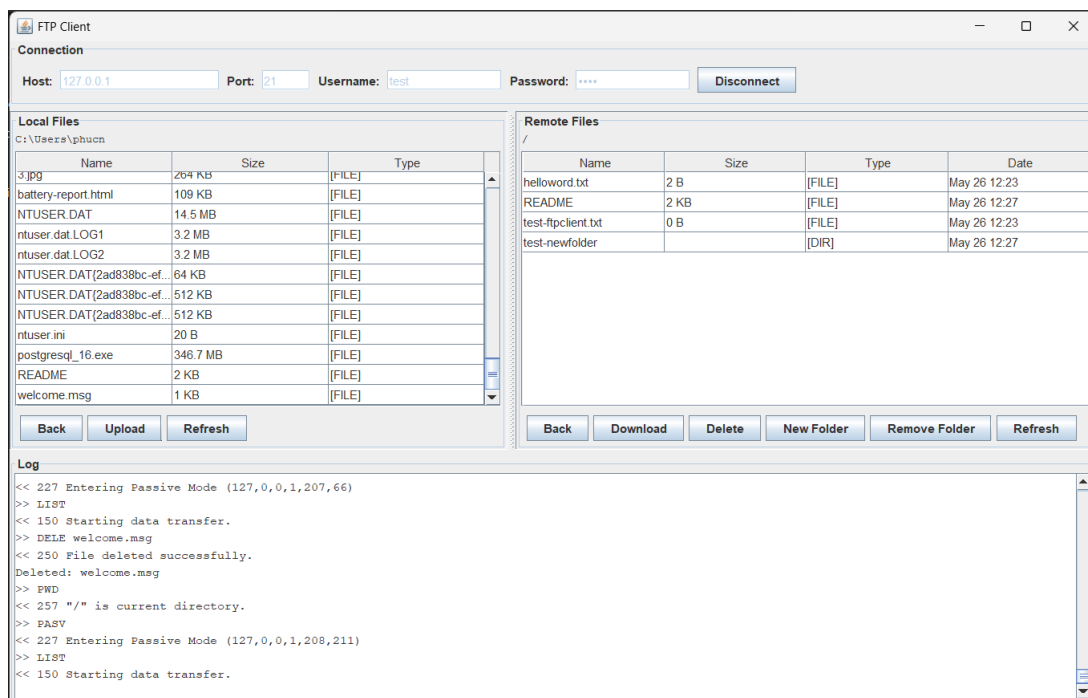


Figure 11: Deleting a remote file

8. I selected the test-newfolder folder in the remote panel and clicked the **Remove Folder** button to delete it.

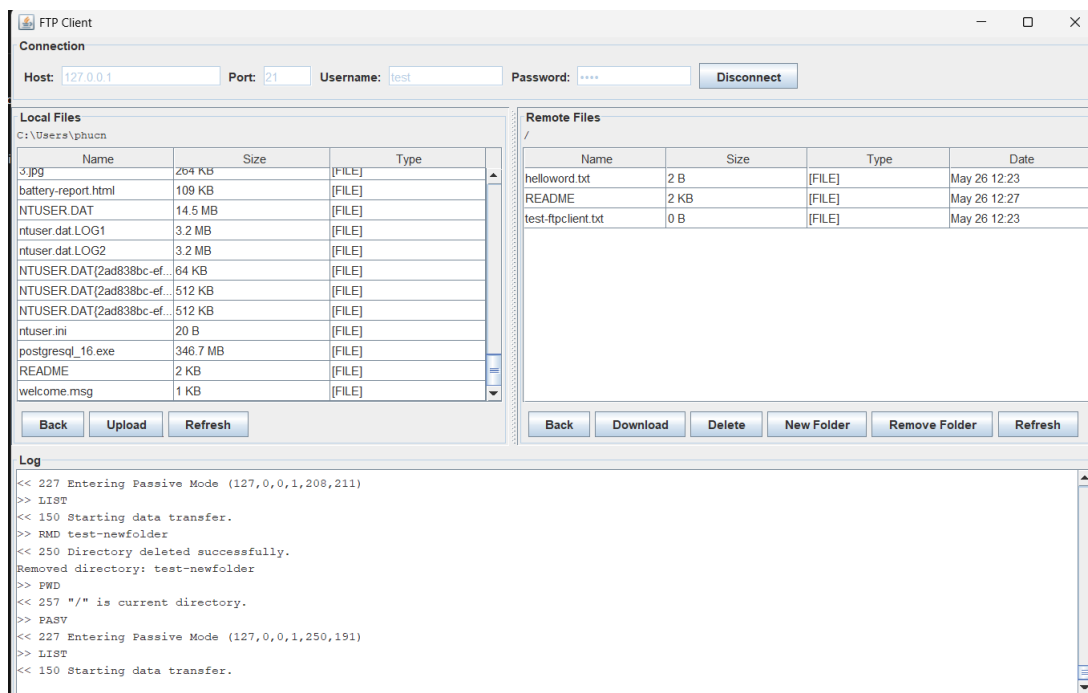


Figure 12: Removing a remote directory

9. I selected the test-ftpclient.txt file in the remote panel and clicked the **Download** button. The file was downloaded and appeared in the local panel.

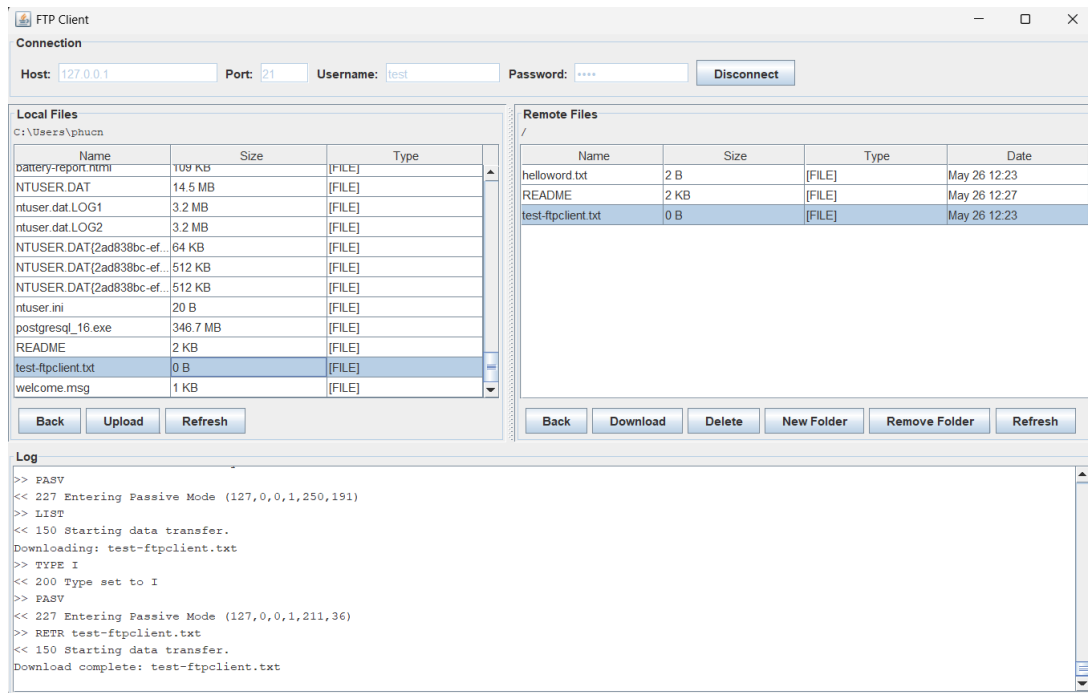


Figure 13: Downloading a file from the local server

10. I clicked the **Disconnect** button to safely terminate the connection.

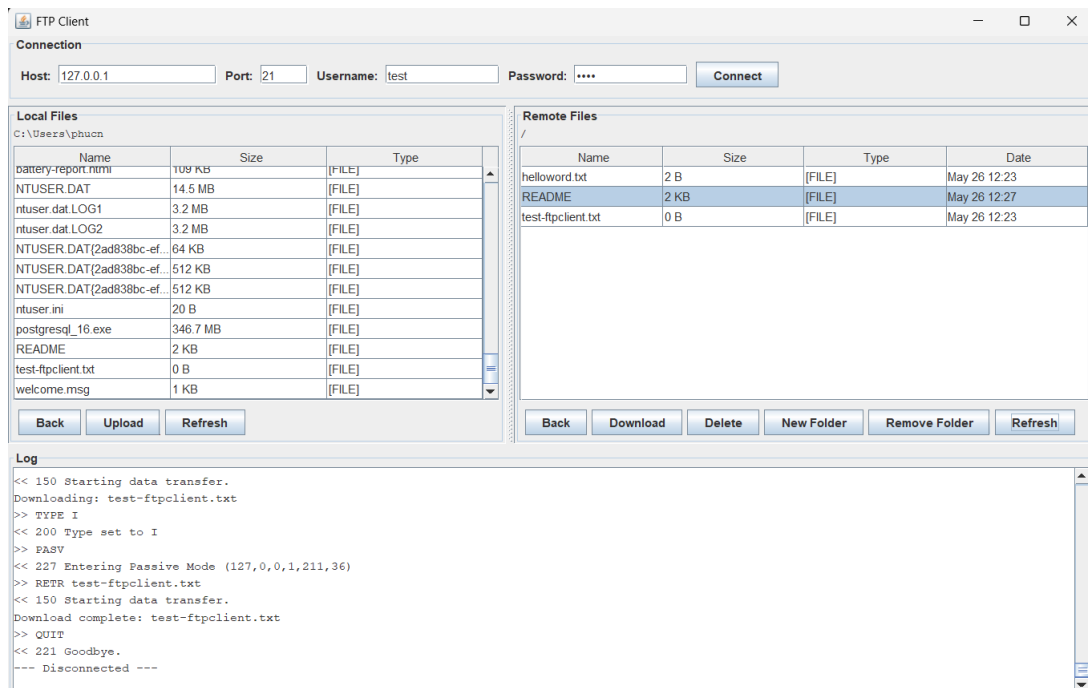


Figure 14: Disconnecting from the local server

5 Conclusion

In conclusion, I have successfully built a fully functional FTP client from scratch using standard Java libraries. By not using any third party libraries, I have learned about TCP socket programming and handling data streams. The application meets all project requirements, it properly handles multiline server responses, works reliably with passive data connections, and provides a simple GUI.

6 Integrity Declaration

I confirm that this submission is my own work. I did not copy code from AI tools, classmates, or online repositories.